

FernUniversität in Hagen
Fakultät für Mathematik und Informatik
LG Kooperative Systeme

UDP Transport Options

Implementierung und Analyse von RFC 9868 in Rust

Masterarbeit
im Studiengang Master Informatik

vorgelegt von
Alan Bernstein
Matrikelnummer: 2068834

Erstprüfer: Prof. Dr. Christian Icking
Zweitprüferin: Dr. Lihong Ma

Hagen, 15.09.2026

Inhaltsverzeichnis

Abbildungsverzeichnis	iv
Tabellenverzeichnis	v
Abkürzungsverzeichnis	1
1 Einleitung	2
1.1 Motivation	2
1.2 Problemstellung	3
1.2.1 Die Surplus Area als ungenutzter Erweiterungsraum	3
1.2.2 Unidirektionalität	3
1.2.3 Implementierungsherausforderungen	4
1.3 Zielsetzung	4
1.4 Abgrenzung und Umfang	6
1.5 Aufbau der Arbeit	6
2 Grundlagen	8
2.1 Internet Protocol (IP)	8
2.1.1 IPv4	8
2.2 Das User Datagram Protocol (UDP)	8
2.2.1 Protokollstruktur	8
2.2.2 Eigenschaften	10
2.3 Middleboxes und Protokoll-Ossifikation	11
2.3.1 Internet-Ossifikation	11
2.3.2 Auswirkungen auf Protokollerweiterungen	11
2.4 RFC 9868: Transport Options for UDP	11
2.4.1 Überblick	11
2.4.2 Die Surplus Area	11
2.4.3 Options-Struktur	12
2.4.4 Definierte Options-Typen	12
2.4.5 SAFE und UNSAFE Options	12
3 Anforderungsanalyse und Technologieauswahl	13
3.1 Anforderungen	13
3.1.1 Funktionale Anforderungen	13
3.1.2 Nicht-funktionale Anforderungen	13
3.1.3 Speichereffizienz und Zero-Copy	13

3.2	Existierende Implementierungen	13
3.2.1	Linux Kernel	13
3.2.2	Userspace-Implementierungen	13
3.3	Herausforderungen	13
3.3.1	Zugriff auf die Surplus Area	13
3.3.2	NAT-Traversal	13
3.3.3	Interoperabilität	13
3.3.4	Alternative Implementierungsansätze	13
3.4	Technologieauswahl	14
3.4.1	Programmiersprache Rust	14
3.4.2	Agentengestützte Entwicklung (Agentic Coding)	20
4	Entwurf und Implementierung	21
4.1	Architektur	21
4.1.1	Architekturüberblick	21
4.1.2	Projektstruktur	21
4.1.3	Datenmodell der UDP Options	21
4.2	TLV-Parser und Serializer	21
4.2.1	Parsing-Algorithmus	21
4.2.2	Serialisierung	21
4.3	Option Checksum (OCS)	21
4.3.1	Berechnung	21
4.3.2	Validierung	21
4.4	Raw-Socket-Zugriff und Plattform-Spezifika	21
4.4.1	Socket-Erstellung und Paketempfang	21
4.4.2	Plattform-Spezifika (Linux und macOS)	21
4.4.3	Build-Konfiguration	22
5	Evaluation	23
5.1	Test- und Messkonzept	23
5.2	Beobachtbarkeit und Middlebox-Verträglichkeit auf realen Pfaden . .	23
5.3	Soll-Ist-Abgleich mit RFC 9868	23
6	Fazit	24
6.1	Beantwortung der Forschungsfragen	24
6.2	Ausblick	24
6.3	Schlusswort	24
	Literaturverzeichnis	25

Abbildungsverzeichnis

Tabellenverzeichnis

3.1	Vergleich der betrachteten Systemprogrammiersprachen anhand der für die RFC 9868-Implementierung maßgeblichen Kriterien.	19
-----	---	----

Abkürzungsverzeichnis

APC Alternate Payload Checksum.

AUTH Authentication Option.

DTLS Datagram Transport Layer Security.

EOL End of List.

FRAG Fragmentation Option.

MDS Maximum Datagram Size.

MRDS Maximum Reassembled Datagram Size.

NOP No Operation.

OCS Option Checksum.

PMTU Path Maximum Transmission Unit.

TIME Timestamp Option.

1. Einleitung

1.1 Motivation

Das User Datagram Protocol (UDP) wurde 1980 in RFC 768 spezifiziert und ist seitdem praktisch unverändert geblieben [1]. Als minimalistisches Transportprotokoll bietet UDP lediglich Portnummern zur Demultiplexierung und eine Prüfsumme zur Fehlererkennung¹. Diese bewusste Einfachheit macht UDP zu einem der am häufigsten genutzten Transportprotokolle im Internet.

Moderne Anwendungen nutzen UDP zunehmend für zeitkritische Kommunikation. Protokolle wie QUIC und WebRTC sowie zahlreiche Spiele- und IoT-Anwendungen setzen auf UDP, um die Latenz des TCP-Handshakes zu vermeiden und mehr Kontrolle über die Übertragungsmechanismen zu erhalten.

Diese Anwendungen stoßen jedoch wiederholt an Grenzen, die UDP allein nicht behandelt: eine Integritätsprüfung über die einfache 16-Bit-Prüfsumme hinaus, eine transportbezogene Fragmentierung unabhängig von IP sowie Mechanismen zum Austausch von Laufzeit- oder MTU-Informationen zwischen Endpunkten. Historisch wurden solche Bedürfnisse entweder den Anwendungen überlassen oder blieben offen.

RFC 9868, veröffentlicht im Oktober 2025, versucht diese Lücke durch die Einführung von *Transport Options for UDP* zu schließen [2]. Die Erweiterung nutzt die sogenannte *Surplus Area* (den möglichen Bereich zwischen dem Ende der UDP-Nutzdaten und dem Ende des IP-Datagramms) für optionale Transportschicht-Mechanismen. Da das UDP-Length-Feld die Länge des UDP-Datagramms (Header und Nutzdaten) angibt und diese kleiner sein kann als die vom IP-Header implizierte Transportnutzlast, entsteht ein bisher ungenutzter Bereich für Erweiterungen.

RFC 9868 definiert ein erweiterbares Options-Framework mit folgenden Eigenschaften:

- Optionen werden im TLV-Format (Type-Length-Value) kodiert und in der *Surplus Area* platziert
- Kategorisierung in SAFE Options (können von Empfängern, die sie nicht kennen, sicher übersprungen werden) und UNSAFE Options (können die Nutzdaten verändern oder ihre Bedeutung ändern und dürfen daher nicht ignoriert werden)
- Integritätsschutz für die Surplus Area über die Option Checksum (OCS); eine Authentifizierungsoption (AUTH) ist als Kennung reserviert

¹ Die Prüfsumme ist optional; wird sie nicht berechnet, wird das Feld auf null gesetzt [1].

- Fragmentierung auf UDP-Ebene, unabhängig von IP-Fragmentierung

Ein entscheidender Vorteil der UDP Options ist ihre **Abwärtskompatibilität**: Bestandssysteme, die UDP Options nicht unterstützen, ignorieren die Surplus Area und verarbeiten lediglich die regulären UDP-Nutzdaten. Dies ermöglicht eine schrittweise Einführung der Erweiterung ohne Beeinträchtigung bestehender Infrastruktur.

Ebenso bleibt UDP unter dem Options-Framework **zustandslos**: UDP Options führen keinen protokolleigenen Zustand ein, da die Zustandshaltung weiterhin der Anwendung obliegt [2, Sec. 6]. Einzig die Zusammenführung fragmentierter Pakete (FRAG) bildet eine eng umgrenzte Ausnahme und erfordert kurzlebigen Empfängerzustand.

1.2 Problemstellung

Die Implementierung von UDP Options gemäß RFC 9868 wirft mehrere technische Herausforderungen auf, die in dieser Arbeit adressiert werden.

1.2.1 Die Surplus Area als ungenutzter Erweiterungsraum

Anders als TCP, dessen Header ein explizites Optionsfeld vorsieht, bietet der ursprüngliche UDP-Header keinen Raum für Erweiterungen. RFC 9868 nutzt stattdessen die *Surplus Area*: den Bereich zwischen dem Ende der durch das UDP-Length-Feld begrenzten Nutzdaten und dem Ende der vom IP-Header implizierten Transportnutzlast. Der genaue Aufbau dieses Bereichs, der mit der *Option Checksum (OCS)* beginnt und die Optionen im TLV-Format aufnimmt, wird in den Grundlagen behandelt (Abschnitt 2.4.2). Für die Problemstellung genügt die Feststellung, dass hier ein bislang ungenutzter, abwärtskompatibler Erweiterungsraum entsteht, dessen Nutzung im Userspace jedoch nicht vorgesehen ist.

1.2.2 Unidirektionalität

RFC 9868 stellt explizit klar: “*UDP is unidirectional; UDP Options do not change that fact.*” [2, Sec. 6]. Diese Design-Entscheidung hat weitreichende Implikationen:

- **Keine Option erfordert eine Antwort:** Keine der in RFC 9868 definierten Optionen verpflichtet den Empfänger zu einer Reaktion. Selbst REQ (Echo Request) erzeugt keine automatische Antwort auf Protokollebene.
- **Antworten sind Anwendungslogik:** Wenn ein Empfänger auf eine REQ-Option mit einer RES-Option (Echo Response) antwortet, geschieht dies durch

eine bewusste Entscheidung der Anwendung, nicht durch einen Protokollmechanismus. Die Anwendung entscheidet, *wann* und *ob* eine Antwort gesendet wird.

Für Anwendungsfälle, die bidirektionale Kommunikation oder garantierte Antworten erfordern, verweist RFC 9868 auf ein noch zu definierendes separates Dokument, das solche Mechanismen spezifizieren soll [2, Sec. 6].² Dies unterstreicht, dass UDP Options bewusst auf der unidirektionalen Natur von UDP aufbauen.

Für die Implementierung bedeutet dies, dass beide Kommunikationspartner als symmetrische Peers agieren: Jeder Peer muss sowohl senden als auch empfangen können, ohne dass das Protokoll eine Client-Server-Hierarchie vorschreibt.

1.2.3 Implementierungsherausforderungen

Die zentrale Herausforderung bei der Implementierung von UDP Options liegt im Zugriff auf die Surplus Area. Existierende Betriebssystem-Stacks liefern über Standard-Socket-APIs ausschließlich die durch das UDP-Length-Feld begrenzten Nutzdaten an die Anwendungsschicht; die Surplus Area wird stillschweigend verworfen [2, Sec. 18]. Da die ursprüngliche UDP-Spezifikation [1] keinen Mechanismus für den Zugriff auf Daten jenseits der UDP-Länge vorsieht, erfordert eine Implementierung im Userspace den Einsatz von Raw Sockets, die vollständige Kontrolle über die Paketstruktur ermöglichen, jedoch erhöhte Systemberechtigungen voraussetzen.

Eine weitere Herausforderung stellt die Kompatibilität mit Middleboxes dar. Während NAT-Geräte UDP-Pakete mit Surplus Area typischerweise korrekt weiterleiten (sie modifizieren lediglich Header-Felder und ignorieren den übrigen IP-Payload), interpretieren bestimmte Intrusion Detection Systeme und Firewalls die Diskrepanz zwischen UDP-Length und IP-Length als Angriffsindikator [2, Sec. 18]. RFC 9868 adressiert dieses Problem durch die Option Checksum (OCS).

1.3 Zielsetzung

Aus der Problemstellung ergeben sich zwei Forschungsfragen, an denen sich Umsetzung und Evaluation dieser Arbeit ausrichten:

- **FF1:** Welche Anforderungen des RFC 9868 lassen sich für eine Userspace-Implementierung über Raw Sockets vollständig, welche nur eingeschränkt und welche gar nicht erfüllen?

² Eine Recherche im IETF Datatracker (Stand Mai 2026) ergab, dass ein solches Dokument noch nicht existiert; weder ein einschlägiger RFC noch ein dedizierter IETF-Draft wurde veröffentlicht.

- **FF2:** In welchem Umfang ist die *Surplus Area* entlang realer Netzpfade zunehmender Realitätsnähe beobachtbar und unverändert zustellbar, und wie verhalten sich NAT-Geräte sowie Middleboxes, die Pakete anhand ihres Inhalts durchlassen oder verwerfen³, gegenüber Paketen mit *Surplus Area*?

Um diese Fragen zu beantworten, entsteht zunächst eine RFC 9868-konforme Bibliothek für UDP Transport Options in der Programmiersprache Rust. Die Implementierung fokussiert auf eine Userspace-Lösung mittels Raw Sockets und umfasst Parser sowie Serializer für TLV-kodierte UDP Options, OCS-Validierung gemäß Section 9 des RFC 9868, Unterstützung der Must-Support Options (EOL, NOP, APC, FRAG, MDS, MRDS, REQ, RES) sowie Raw-Socket-Programmierung für den Zugriff auf die *Surplus Area*.

Die Verifikation der Implementierung erfolgt mehrstufig: Unit Tests prüfen die einzelnen Komponenten, Integrationstests prüfen den Loopback-Pfad, und für die reinen, speicher- und systemeffektfreien Wire-Regeln wird ergänzend eine Lean-Spezifikation gepflegt. Lean dient dabei als formale Gegenprobe für modellierbare Invarianten wie Checksum-Arithmetik, Längen- und Layoutregeln; Raw-Socket-Verhalten, Betriebssystemeffekte und Middlebox- Verhalten bleiben empirische Prüfpunkte. Darauf aufbauend wird die Bibliothek in mehreren Umgebungen mit zunehmender Realitätsnähe erprobt: einer lokal virtualisierten Umgebung, einer tunnelbasierten Kopplung zweier Endpunkte, einer Verbindung zwischen zwei Hosts innerhalb Deutschlands sowie einer interkontinentalen Verbindung. Der Vergleich dieser Umgebungen erlaubt Aussagen darüber, inwieweit Annahmen aus kontrollierten Testbedingungen auf reale Netzpfade übertragbar sind, und beantwortet damit FF2.

Auf dieser Grundlage erfolgt eine Evaluierung der praktischen Implementierung hinsichtlich Funktionsfähigkeit, Beobachtbarkeit der *Surplus Area* entlang des Pfades sowie Kompatibilität mit bestehender Netzinfrastruktur (insbesondere NAT-Geräten, Firewalls und IDS/IPS-Systemen). Abschließend werden Spezifikation und tatsächliche Umsetzung gegenübergestellt: Welche Anforderungen des RFC 9868 konnten vollständig, welche nur eingeschränkt und welche nicht im Userspace umgesetzt werden, und welche Diskrepanzen zwischen Soll- und Ist-Verhalten ergeben sich aus den Beschränkungen der eingesetzten Schnittstellen. Dieser Soll-Ist-Abgleich beantwortet FF1.

Der Beitrag dieser Arbeit ist damit zweigeteilt und in beiden Teilen gleichgewichtig: Zum einen entsteht eine quelloffene, RFC 9868-konforme Referenzimplementierung der Kern-Mechanismen für den Userspace; zum anderen liefert die abgestufte Erprobung einen empirischen Befund darüber, wie weit sich diese frisch standardisierte Erweiterung über reale Internetpfade hinweg tatsächlich einsetzen

³ Etwa Firewalls, DPI- sowie IDS/IPS-Systeme und Paketnormalisierer.

lässt. Die Bibliothek stellt das notwendige Instrument bereit, während der eigenständige Erkenntnisgewinn vor allem im empirischen Teil liegt, der über eine reine Implementierung hinausgeht.

1.4 Abgrenzung und Umfang

Diese Arbeit setzt einen bewussten Rahmen. Sie beschränkt sich auf den im Userspace realisierbaren Kern des RFC 9868: das TLV-Options-Framework, die *Option Checksum (OCS)* sowie die Must-Support Options. Die Beschränkung auf den Userspace ist eine vorab gesetzte Rahmenbedingung der Aufgabenstellung. Die konkrete Mechanik des Zugriffs (Raw Sockets gegenüber kernelnahen Verfahren) wird dabei nicht vorweggenommen, sondern erst in Abschnitt 3.4 abgewogen und begründet.

Explizit außerhalb des Umfangs dieser Arbeit liegen der REQ/RES-Anwendungsfall für Path MTU Discovery, welcher im Nachfolge-RFC 9869 (DPLPMTUD for UDP Options) definiert ist [3], die TIME-Option, für die RFC 9868 zwar das Optionsformat und den Anwendungszweck (RTT-Schätzung) beschreibt, aber kein konkretes nutzendes Protokoll festlegt, sowie die Optionen AUTH, UCMP und UENC, deren Optionstypen (*Kind*) in RFC 9868 bisher nur reserviert sind (AUTH [2, Sec. 11.9], UCMP [2, Sec. 12.1], UENC [2, Sec. 12.2]). Kernel-Module werden nicht entwickelt.

Ebenfalls nicht betrachtet wird IPv6. Der Mechanismus des RFC 9868 (*Surplus Area*, OCS, TLV-Optionen und FRAG) ist gegenüber der IP-Version neutral und lässt sich vollständig über IPv4 zeigen. Der IPv6-spezifische Anteil beschränkt sich im Wesentlichen auf die abweichende Semantik des Raw-Socket-Zugriffs, die zusätzliche Plattformabhängigkeiten einbringt, ohne zum Erkenntnisgewinn über das Options-Framework beizutragen. Alle Implementierungs- und Messergebnisse dieser Arbeit beziehen sich daher auf IPv4.

1.5 Aufbau der Arbeit

Die Arbeit gliedert sich in sechs Kapitel. Kapitel 2 führt die technischen Grundlagen ein: das Internet Protocol, UDP, Middleboxes und Protokoll-Ossifikation sowie RFC 9868 mit der *Surplus Area*. Kapitel 3 analysiert die Anforderungen, sichtet bestehende Implementierungen, arbeitet die Herausforderungen heraus und trifft die Technologie- und Architekturentscheidung. Kapitel 4 führt Entwurf und Implementierung der Bibliothek komponentenweise zusammen. Kapitel 5 berichtet die funktionale und formale Verifikation sowie die Erprobung auf den vier Netzpfaden. Kapitel 6 fasst die Ergebnisse entlang der Forschungsfragen zusammen.

Diese lineare Kapitelfolge ist eine rekonstruktive Darstellungsstruktur und kein Abbild des tatsächlichen Arbeitsprozesses. Die Umsetzung verlief iterativ und agenten-

gestützt, sodass Entwurf, Implementierung und Test einander fortlaufend bedingen. Die zur Analyse eingesetzten Werkzeuge (Wireshark und tcpdump) sowie die methodische Rolle der eingesetzten Sprachmodelle werden dort behandelt, wo sie wirksam werden: bei der Technologie- und Methodenwahl in Abschnitt 3.4.2.

2. Grundlagen

2.1 Internet Protocol (IP)

2.1.1 IPv4

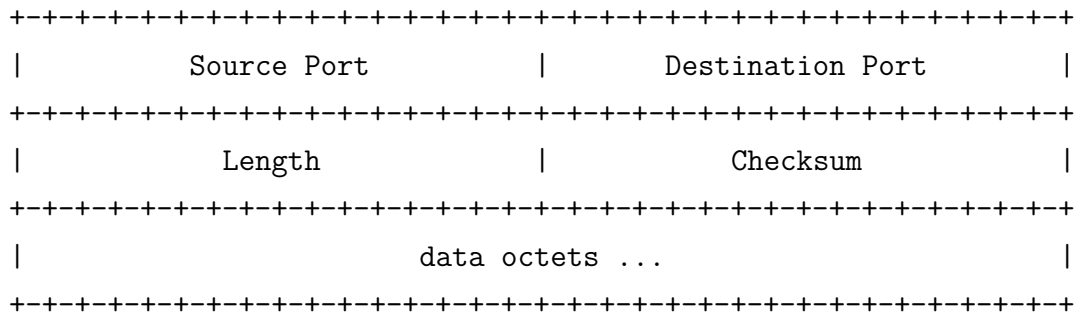
2.2 Das User Datagram Protocol (UDP)

Das User Datagram Protocol (UDP) ist neben dem Transmission Control Protocol (TCP) eines der beiden zentralen Transportprotokolle der TCP/IP-Protokollfamilie. Im Gegensatz zu TCP verfolgt UDP ein bewusst minimalistisches Design: Es sendet Datagramme ohne vorherigen Handshake oder Zustandsabgleich mit dem Empfänger, bietet keine Garantien für Zustellung, Reihenfolge oder Duplikaterkennung und verzichtet auf jeglichen Flusskontroll- oder Staukontrollmechanismus [1]. Mit der Protokollnummer 17 im IP-Header registriert, wurde UDP in RFC 768 spezifiziert und trägt die Bezeichnung STD 6 als Internet-Standard. Bemerkenswert ist, dass RFC 768 seit seiner Veröffentlichung im August 1980 über 45 Jahre hinweg keine formelle Aktualisierung erfahren hat. Erst RFC 9868 trägt den Header-Eintrag **Updates: 768** und erweitert damit den ursprünglichen UDP-Standard erstmalig direkt [2].

Historisch geht UDP auf die Aufspaltung des ursprünglichen *Transmission Control Program* zurück, das Cerf, Dalal und Sunshine Ende 1974 noch als monolithische Einheit aus Netzwerk- und Transportfunktionen beschrieben [4]. Mit deren Trennung in IP und TCP entstand der Bedarf nach einer schlanken, transaktionsorientierten Alternative ohne den Overhead von TCP, die Jon Postel 1980 spezifizierte [1]. Während der UDP-Kern über die folgenden Jahrzehnte unverändert blieb und nur Begleitdokumente Randfragen klärten, ließ erst die wachsende Bedeutung UDP-basierter Protokolle wie QUIC [5] und HTTP/3 [6] mit RFC 9868 die erste direkte Erweiterung des Standards entstehen.

2.2.1 Protokollstruktur

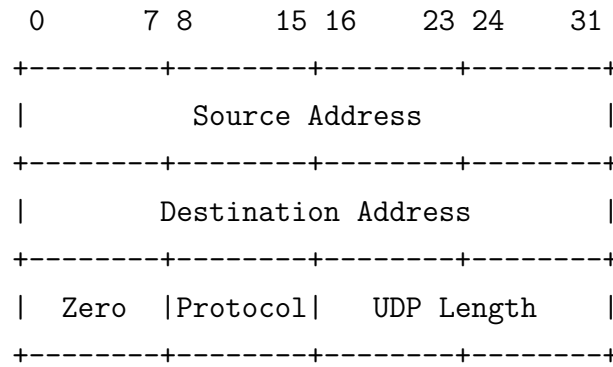
Ein UDP-Datagramm besteht aus einem Header mit fester Länge von 8 Byte und den darauf folgenden Nutzdaten. RFC 768 definiert den Header wie folgt [1]:



Die vier Felder des Headers haben jeweils eine Breite von 16 Bit:

- **Source Port** – Die Portnummer des sendenden Prozesses. Dieses Feld ist optional; wird es nicht benötigt, wird es auf null gesetzt [1]. In der Praxis nutzen Anwendungen den *Source Port* zur Identifikation des Absenders, damit der Empfänger Antworten an den korrekten Port zurücksenden kann.
- **Destination Port** – Die Portnummer des Zielprozesses auf dem empfangenden Host. Zusammen mit der IP-Zieladresse ermöglicht dieses Feld die *Demultiplexierung* eingehender Datagramme an die korrekte Anwendung.
- **Length** – Die Gesamtlänge des UDP-Datagramms in Byte, bestehend aus Header *und* Nutzdaten. Der Minimalwert beträgt 8, was einem Datagramm ohne Nutzdaten entspricht [1]. Da das Feld 16 Bit breit ist, ergibt sich eine theoretische Maximalgröße von 65 535 Byte.
- **Checksum** – Eine Prüfsumme über einen Pseudo-Header, den UDP-Header und die Nutzdaten. Die Berechnung verwendet das *Einerkomplement*-Verfahren: Alle 16-Bit-Wörter werden addiert und das Einerkomplement der Summe bildet den Prüfsummenwert [7]. Die Prüfsumme ist optional: wird sie nicht berechnet, wird das Feld auf null gesetzt [1].

Pseudo-Header. Die UDP-Prüfsumme schützt nicht nur die Transportschichtdaten, sondern bezieht über einen *Pseudo-Header* auch Informationen der Vermittlungsschicht ein. Dieses Konzept stellt sicher, dass ein Datagramm, das durch einen IP-Fehler an den falschen Host oder das falsche Protokoll zugestellt wird, erkannt und verworfen werden kann. RFC 768 definiert den folgenden Pseudo-Header [1]:



2.2.2 Eigenschaften

Die in Abschnitt 2.2.1 beschriebene Header-Struktur verdeutlicht bereits das minimalistische Design von UDP. Aus diesem Designziel leiten sich eine Reihe funktionaler Eigenschaften ab, die maßgeblich beeinflussen, für welche Anwendungsfälle das Protokoll geeignet ist.

Nachrichtenorientierung. UDP ist ein *nachrichtenorientiertes* Protokoll: Jede Sendeoperation erzeugt genau ein Datagramm, und der Empfänger erhält jede Nachricht als atomare Einheit. RFC 768 beschreibt UDP als Verfahren, mit dem Programme Nachrichten an andere Programme senden können („a procedure to send [...] messages to other programs“) [1].

Zustandslosigkeit. UDP verwaltet keinen protokollspezifischen Zustand für einzelne Kommunikationsbeziehungen und hält keinerlei Datenstrukturen wie Sequenznummern, Fenstergrößen oder Retransmission-Timer vor [2, Sec. 6].

Minimale Latenz. Da UDP verbindungslos arbeitet, entfällt jeglicher Handshake vor der Datenübertragung. Die erste Nachricht kann unmittelbar gesendet werden, ohne dass ein vorheriger Verbindungsaufbau abgewartet werden muss [8, Sec. 1]. Ebenso existiert kein Verbindungsabbau, der zusätzliche Latenz verursachen könnte.

Unabhängigkeit der Datagramme. Jedes UDP-Datagramm ist eine eigenständige, von allen anderen Datagrammen unabhängige Einheit. Der Verlust eines Datagramms hat daher keine Auswirkung auf die Verarbeitung nachfolgender Datagramme. Diese Eigenschaft vermeidet das sogenannte *Head-of-Line-Blocking* (HoL-Blocking), bei dem verlorene Pakete die Auslieferung nachfolgender Daten verzögern. Diese Eigenschaft von UDP war eine der Motivationen für die Entwicklung von QUIC: QUIC nutzt UDP als Transportschicht und realisiert auf Anwendungsebene ein Stream-Multiplexing, bei dem der Verlust eines Pakets nur den betroffenen Stream blockiert, nicht jedoch andere parallele Streams [5, Sec. 1].

Multicast und Broadcast. UDP ist verbindungslos und wird häufig als Transport für IP-Multicast und IP-Broadcast verwendet. Ein UDP-Datagramm kann an eine Multicast-Gruppe oder eine Broadcast-Adresse gesendet und von mehreren Empfängern empfangen werden, sofern Netzwerk und Hosts entsprechend konfiguriert sind. Dabei gelten die üblichen Eigenschaften von UDP: keine Garantie für Zustellung, Reihenfolge oder Duplikaterkennung. Diese Kombination eignet sich besonders für lokale Mechanismen wie Service Discovery sowie für bestimmte Echtzeitanwendungen in kontrollierten Netzen.

Diese Eigenschaften machen UDP zu einem attraktiven Transportprotokoll für latenzempfindliche, verlusttolerante oder gruppenbasierte Anwendungen. Allerdings fehlt UDP jede Form der protokolleigenen Erweiterbarkeit. Genau diese Lücke adressiert RFC 9868 durch die Nutzung der *Surplus Area* (siehe Abschnitt 2.4.2), um Optionen zu transportieren, ohne den bestehenden UDP-Header zu verändern.

2.3 Middleboxes und Protokoll-Ossifikation

2.3.1 Internet-Ossifikation

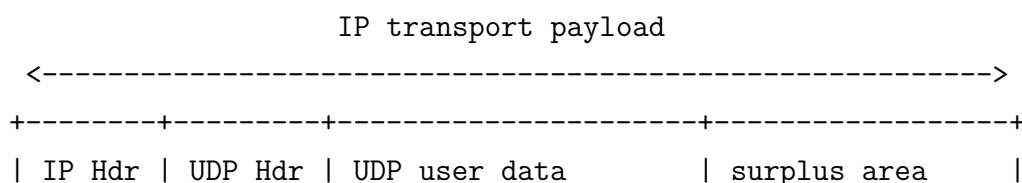
2.3.2 Auswirkungen auf Protokollerweiterungen

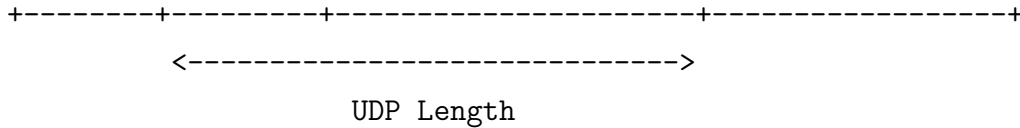
2.4 RFC 9868: Transport Options for UDP

2.4.1 Überblick

2.4.2 Die Surplus Area

Da das UDP-Length-Feld die Länge des UDP-Datagramms (Header und Nutzdaten) angibt und diese kleiner sein kann als die vom IP-Header implizierte Transportnutzlast, entsteht zwischen dem Ende der UDP-Nutzdaten und dem Ende des IP-Datagramms ein bislang ungenutzter Bereich. RFC 9868 bezeichnet ihn als *Surplus Area* und verwendet ihn zur Aufnahme der UDP Options [2]:





Die *Surplus Area* beginnt mit der *Option Checksum (OCS)*, gefolgt von den eigentlichen Optionen im TLV-Format (Abschnitt 2.4.3). Diese Struktur ermöglicht variable Optionslängen und zukünftige Erweiterungen. Da Empfänger, die UDP Options nicht unterstützen, ausschließlich die durch das UDP-Length-Feld begrenzten Nutzdaten verarbeiten, bleibt die Erweiterung abwärtskompatibel.

Wesentlich für die Einordnung ist, dass RFC 9868 kein neues Verhalten der Vermittlungsschicht einführt: Es definiert weder neue IP-Header-Felder noch verändert es die Verarbeitung von IPv4. Für die IP-Schicht sind die Bytes der *Surplus Area* reine Transportnutzlast innerhalb der durch IP Total Length begrenzten Datagramm-Größe; sie liegen lediglich hinter dem durch UDP Length markierten Ende der UDP-Nutzdaten. RFC 9868 ist damit ein reines Transport-Feature, das ausschließlich die zuvor ungenutzte Differenz zwischen UDP Length und der IP-Transportnutzlast belegt [2].

2.4.3 Options-Struktur

2.4.4 Definierte Options-Typen

2.4.5 SAFE und UNSAFE Options

3. Anforderungsanalyse und Technologieauswahl

3.1 Anforderungen

3.1.1 Funktionale Anforderungen

3.1.2 Nicht-funktionale Anforderungen

3.1.3 Speichereffizienz und Zero-Copy

3.2 Existierende Implementierungen

3.2.1 Linux Kernel

3.2.2 Userspace-Implementierungen

3.3 Herausforderungen

3.3.1 Zugriff auf die Surplus Area

3.3.2 NAT-Traversal

3.3.3 Interoperabilität

3.3.4 Alternative Implementierungsansätze

3.4 Technologieauswahl

Die vorangegangene Analyse hat den Rahmen definiert, innerhalb dessen die Implementierung zu realisieren ist. Dieser Abschnitt überführt diese Analyse in die tragenden Technologie- und Methodenentscheidungen und begründet sie entlang der herausgearbeiteten Anforderungen.

3.4.1 Programmiersprache Rust

Die Wahl der Implementierungssprache wird nicht vorausgesetzt, sondern als Konsequenz der zuvor formulierten funktionalen und nicht-funktionalen Anforderungen entwickelt. Der Argumentationsbogen verläuft dabei in mehreren Schritten: von den Anforderungen, die das Vorhaben an eine geeignete Programmiersprache stellt, über die begriffliche Einordnung in die Klasse der Systemprogrammiersprachen und die Begründung der konkreten Sprachwahl bis zu einem Vergleich mit den Alternativen.

Aus dem RFC 9868 ergeben sich mehrere Anforderungen an die Programmiersprache, noch bevor eine konkrete Sprache benannt wird. Sie leiten sich unmittelbar aus den nicht-funktionalen Anforderungen ab. Zunächst wird ein direkter Zugriff auf *Raw Sockets* und damit auf rohe IP-Datagramme benötigt, da die Optionen von RFC 9868 in der *Surplus Area* jenseits der regulären UDP-Nutzdaten liegen und nur über selbst konstruierte Datagramme mit eigenem IP-Header (etwa mittels `IP_HDRINCL`) vollständig kontrolliert werden können. Hinzu kommt der Bedarf an einer bytengenauen Kontrolle in der *Surplus Area*, da die TLV-kodierten Optionen exakt an den durch den RFC vorgegebenen Bytegrenzen ausgerichtet sein müssen und jede implizite Umordnung oder Auffüllung durch die Sprache oder ihre Laufzeitumgebung die Korrektheit gefährden würde. Ebenso müssen potentiell bösartige eingehende Pakete ohne nennenswerten Laufzeit-Overhead verarbeitet werden, denn der Parser bildet die Vertrauensgrenze gegenüber dem Netz und darf weder durch präparierte Längfelder zum Absturz gebracht noch durch defensive Schutzmechanismen verlangsamt werden. Schließlich wird ein deterministisches Speicherverhalten ohne Garbage Collector vorausgesetzt, da solche Pausen das Laufzeitverhalten nicht-deterministisch machen würden. Diese Anforderungen bilden die Voraussetzungen zur Auswahl einer geeigneten Programmiersprache.

Sprachen, die diese Anforderungen erfüllen können, werden üblicherweise als *Systemprogrammiersprachen* zusammengefasst. Kennzeichnend für diese Klasse ist hardwarenahe Kontrolle über Speicher und Ausführung: direkter Zugriff auf Speicheradressen, Bitmuster und Betriebssystemschnittstellen ohne vermittelnde Abstraktionsschicht. Hinzu kommt eine explizite Speicherverwaltung ohne obligatorische

Laufzeitumgebung und insbesondere ohne Garbage Collector, sodass Allokation und Freigabe dem Programm und nicht einem im Hintergrund laufenden Prozess obliegen; der Quelltext wird zu nativem Maschinencode übersetzt, der ohne zwischengeschaltete virtuelle Maschine ausgeführt wird, woraus ein weitgehend deterministisches Laufzeitverhalten folgt. Eben diese Eigenschaften prädestinieren die Klasse für Aufgaben mit unmittelbarem Hardware- und Systembezug wie die Entwicklung von Betriebssystemen, Gerätetreibern und Netzwerkstacks. Genau in diese Schicht fällt die Implementierung eines Transportprotokoll-Mechanismus.

Die Definition wird von einer überschaubaren Gruppe etablierter Sprachen erfüllt, insbesondere von C, C++, Zig und Rust. Alle genannten Kandidaten bieten den geforderten hardwarenahen Zugriff und die Kompilation zu nativem Code; sie unterscheiden sich jedoch grundlegend darin, welche Sicherheitsgarantien sie dabei gewähren. Das entscheidende Kriterium ergibt sich daher nicht aus der Sprachklasse selbst, sondern aus der folgenden Anforderung: der sicheren Verarbeitung potentiell bösartiger Eingaben. Da der Parser die Vertrauensgrenze gegenüber dem Netz bildet, wird die standardmäßige Speichersicherheit zum entscheidenden Auswahlkriterium erhoben, denn eine Sprache, die Speicherfehler bereits konstruktiv ausschließt, verlagert eine ganze Klasse sicherheitskritischer Fehler von der Laufzeit in die Übersetzungsphase.

Dass diese Fehlerklasse kein akademisches Detail, sondern den weitaus größten Teil schwerwiegender Schwachstellen ausmacht, ist empirisch breit belegt: Sowohl bei Microsoft als auch im Chromium-Projekt werden übereinstimmend rund 70 Prozent der schwerwiegenden Sicherheitslücken auf Speichersicherheitsfehler zurückgeführt [9, 10], weshalb selbst die US-amerikanische National Security Agency (NSA) den Umstieg auf speichersichere Sprachen ausdrücklich empfiehlt und dabei Rust namentlich nennt [11]. Legt man die standardmäßige Speichersicherheit als Auswahlkriterium an das Feld an, so bleibt unter den etablierten Systemprogrammiersprachen Rust als einzige übrig, die diese Garantie ohne Garbage Collector und bereits zur Übersetzungsphase erbringt; die Wahl fällt folglich auf **Rust**.

Rust ist eine kompilierte, statisch und stark typisierte Systemprogrammiersprache, die ohne Garbage Collector und ohne obligatorische Laufzeitumgebung auskommt [12, 13]. Die statische Typisierung bedeutet dabei, dass die Typen aller Werte bereits zur Übersetzungszeit feststehen und vom Compiler geprüft werden, sodass eine ganze Klasse von Typfehlern vor der Ausführung und ohne Laufzeitkosten ausgeschlossen wird. Zugleich besteht über ein C-kompatibles *Foreign Function Interface* ein direkter Zugang zu den POSIX-Systemaufrufen, über die die benötigten *Raw Sockets* eingerichtet und die in der ersten Anforderung genannten IP-Datagramme kontrolliert

werden. Damit erfüllt Rust sämtliche Merkmale der zuvor entwickelten Definition: Es bietet hardwarenahe Kontrolle, verzichtet auf eine obligatorische Laufzeitumgebung und erzeugt deterministisch ausführbaren nativen Code.

Charakteristisch für Rust ist eine bewusste Auswahl der vorhandenen wie der absichtlich nicht vorhandenen Sprachmittel. Vorhanden sind das im nächsten Block ausführlich behandelte System aus *Ownership* und *Borrowing*; ferner algebraische Datentypen in Gestalt von `enum` mit vom Compiler auf Vollständigkeit geprüftem *Pattern Matching*; eine auf dem Typ `Result` aufbauende explizite Fehlerbehandlung; eine starke statische Typisierung; sowie das Schlüsselwort `unsafe` als bewusst markierte und eng begrenzte Brücke für jene Operationen, deren Sicherheit der Compiler nicht selbst nachweisen kann. Nicht vorhanden sind hingegen ein `null`-Wert, implizite Typkonvertierungen, klassische *Exceptions* sowie die Implementierungsvererbung objektorientierter Sprachen.

Gerade das Fehlen dieser Konstrukte verringert die Zahl der Fehlerquellen in einem sicherheitskritischen Parser erheblich. Diese Sprachmittel prägen den Aufbau der vorliegenden Implementierung unmittelbar. Sämtliche `unsafe`-Blöcke sind in der Socket-Schicht (`src/socket/`) konzentriert und dort hinter sichere Wrapper gekapselt, sodass der übrige Code ausschließlich mit sicheren Abstraktionen arbeitet.

Algebraische Datentypen und *Pattern Matching* bilden die TLV-kodierten Optionen der *Surplus Area* typsicher und erschöpfend ab, und die `Result`-basierte Fehlerbehandlung zwingt dazu, fehlerhafte oder bösartige Eingaben an jeder Verarbeitungsstelle explizit zu behandeln. Parser, Serializer und Prüfsummenberechnung sind dabei bewusst handgeschrieben und greifen nicht auf etablierte Bibliotheken wie `nom` oder `zerocopy` zurück, da die Mechanik von RFC 9868 selbst Gegenstand der Untersuchung ist und durch eine fertige Abstraktion nicht verdeckt werden soll.

Den Kern der Sprachwahl bildet die Art und Weise, in der Rust *Memory Safety* erzwingt: zum überwiegenden Teil bereits zur Übersetzungszeit durch das Typsystem, ergänzt um wenige, eng umrissene Laufzeitprüfungen. Unter *Memory Safety* wird die Abwesenheit einer Klasse von Speicherzugriffsfehlern verstanden, zu der insbesondere der *Buffer Overflow* (ein Zugriff jenseits der Grenzen eines reservierten Speicherbereichs), das *Use-after-free* (ein Zugriff auf bereits freigegebenen Speicher) und der *Data Race* (ein unsynchronisierter, nebenläufiger Zugriff mehrerer Ausführungsstränge, von denen mindestens einer schreibt) zählen. Rust begegnet diesen Fehlerklassen auf zwei Wegen: Das *Use-after-free* und der *Data Race* werden durch ein statisches Regelwerk ausgeschlossen, das aus drei aufeinander aufbauenden Konzepten besteht und das der Compiler vor der Codeerzeugung durchsetzt [13, 14]; der Zugriff jenseits der Grenzen eines Speicherbereichs hingegen wird, soweit er nicht ohnehin statisch

ausgeschlossen ist, durch automatisch eingefügte Bereichsprüfungen abgefangen, die bei einer Verletzung einen kontrollierten Programmabbruch (*panic*) auslösen, statt wie in C undefiniertes Verhalten zuzulassen [13, Kap. 9].

Das erste ist *Ownership*: Jeder Wert besitzt zu jedem Zeitpunkt genau einen Eigentümer; wird der Wert einer anderen Bindung zugewiesen oder an eine Funktion übergeben, so geht das Eigentum über (*move*), und die ursprüngliche Bindung ist anschließend nicht mehr verwendbar, was ein doppeltes Freigeben desselben Speichers ausschließt; verlässt der Eigentümer seinen Gültigkeitsbereich, wird der zugehörige Speicher deterministisch freigegeben, ohne dass es dafür eines Garbage Collectors bedürfte.

Das zweite ist *Borrowing*: Anstelle einer Eigentumsübertragung kann ein Wert über Referenzen geborgt werden, wobei zu jedem Zeitpunkt entweder beliebig viele geteilte, ausschließlich lesende Referenzen oder genau eine exklusive, schreibende Referenz bestehen dürfen, niemals jedoch beides zugleich; diese Regel schließt *Data Races* konstruktiv aus, da ein unsynchronisierter schreibender Zugriff bei gleichzeitig bestehenden weiteren Referenzen nicht typisierbar ist.

Das dritte sind *Lifetimes*: Sie binden die Gültigkeitsdauer jeder Referenz an jene des referenzierten Wertes und stellen so sicher, dass keine Referenz ihren Referenten überdauert, womit *Use-after-free* ausgeschlossen wird.

Die Gesamtheit dieser Regeln erzwingt der *Borrow Checker* vollständig zur Übersetzungszeit: Code, der gegen eine der Regeln verstößt, wird nicht übersetzt, und es entstehen keinerlei Laufzeitkosten für diese Prüfung. Das nachstehende Beispiel zeigt den Mechanismus an einem minimalen Fall, in dem eine geteilte und eine exklusive Referenz koexistieren sollen.

```
1 let mut buf = vec![1u8, 2, 3];
2 let view = &buf[0];           // geteilte (lesende) Referenz auf buf
3 buf.push(4);                  // Fehler: exklusiver Borrow, solange '
    view' lebt
4 println!("{view}");           // Nutzung von 'view' haelt den Borrow
    am Leben
```

Der Borrow Checker verweigert hier die Übersetzung mit dem Fehlercode E0502, weil `buf.push` eine exklusive Referenz auf den Puffer benötigt, während die geteilte Referenz `view` noch gebraucht wird; ein in C oder C++ an dieser Stelle möglicher Zugriff über einen durch die Reallokation ungültig gewordenen Zeiger ist damit bereits zur Übersetzungszeit ausgeschlossen:

```
1 --> src/main.rs:4:1
```

```

2 |
3 | let view = &buf[0];          // geteilte (lesende) Referenz auf
  | buf
4 |          --- immutable borrow occurs here
5 | buf.push(4);                // Fehler: exklusiver Borrow,
  | solange 'view' lebt
6 | ~~~~~ mutable borrow occurs here
7 | println!("{view}");         // Nutzung von 'view' haelt den
  | Borrow am Leben
8 |          ---- immutable borrow later used here
9 |
10 | For more information about this error, try 'rustc --explain E0502'.

```

Darauf aufbauend lässt sich die Wahl gegen die übrigen Kandidaten der Klasse präzise begründen, ohne deren jeweilige Stärken zu übergehen.

C bietet maximale, unverstellte Kontrolle über Speicher und Hardware und besitzt das mit Abstand reife Ökosystem für hardwarenahe Netzprogrammierung; diese Kontrolle wird jedoch ohne jede sprachseitige Sicherheitsgarantie gewährt, denn weder Schranken- noch Lebensdauerprüfungen für Zeiger sind Bestandteil der Sprachdefinition, und der Standard kennt eine große Zahl von Fällen mit *Undefined Behavior*, darunter gerade Pufferüberläufe und Zugriffe auf freigegebenen Speicher [15].

C++ erweitert dieses Fundament um mächtige Abstraktionsmittel und mildert mit *RAII* sowie Smart Pointern einen Teil der manuellen Speicherverwaltung, bleibt jedoch unsicher per Voreinstellung, da diese Werkzeuge optional sind und neben den unsicheren C-Altlasten fortbestehen, und erkaufte seine Ausdruckstärke mit erheblicher Sprachkomplexität.

Zig ist eine moderne, bewusst einfach und explizit gehaltene Systemprogrammiersprache, die mit *comptime* eine ausdrucksstarke Auswertung zur Übersetzungszeit bietet und versteckte Allokationen vermeidet, indem Speicher ausschließlich über explizit übergebene Allocator-Objekte angefordert wird [16]; eine compilergarantierte Speichersicherheit nach Art des Borrow Checkers kennt Zig jedoch nicht, sondern erkennt nur eine Teilmenge unzulässiger Zugriffe durch Laufzeitprüfungen in den sicheren Build-Modi, die in den auf Geschwindigkeit oder Größe optimierten Modi entfallen, und hat mit Version 0.16.0 noch keine stabile Fassung erreicht [16]¹.

Rust nimmt in diesem Feld die Stellung ein, C-vergleichbare Kontrolle und Performance mit einer bereits zur Übersetzungszeit erzwungenen Speicher- und Data-

¹ Welche praktische Relevanz dieser Unterschied besitzt, zeigt die JavaScript-Laufzeit Bun, lange das größte Produktivprojekt in Zig: Nachdem allein in Version 1.3.14 vierzig Speicherlecks und zahlreiche Abstürze einzeln behoben worden waren, portierte das Team um Jarred Sumner den Kern von Zig nach Rust, um derartige Fehler strukturell auszuschließen, statt sie dauerhaft nachzubessern [17].

Race-Freiheit zu verbinden; der Preis dafür ist die Einarbeitung in das Ownership-Modell, die jedoch durch präzise Compiler-Diagnosen unterstützt wird.

Abschließend stellt Tabelle 3.1 die entscheidungsrelevanten Eigenschaften der vier Kandidaten gegenüber.

Tabelle 3.1: Vergleich der betrachteten Systemprogrammiersprachen anhand der für die RFC 9868-Implementierung maßgeblichen Kriterien.

Kriterium	C	C++	Zig	Rust
Speichersicherheit standardmäßig	nein	nein	nur in sicheren Build-Modi	✓ (Compiler)
Data-Race-Freiheit zur Übersetzungszeit	nein	nein	nein	✓
Speicherverwaltung	manuell	manuell, RAII	manuell, explizite Allocator	Ownership, kein GC
Laufzeit-Overhead	keiner, da ungeprüft	keiner, da ungeprüft	nur in sicheren Build-Modi	minimal (Be- reichsprüfung)
Compiler-Diagnostik als Rückkopplungsschleife	begrenzt	begrenzt, oft unübersichtlich	solide	ausgeprägt, mit stabilen Fehlercodes
Reife des Ökosystems (hardwarenahe Netze)	sehr hoch	sehr hoch	gering, noch nicht stabil	hoch, wachsend

Über die reinen Sicherheitsgarantien hinaus erweist sich der Rust-Compiler als eine starke, deterministische und maschinenlesbare Rückkopplungsschleife, was im Rahmen dieser Arbeit aus einem zweiten Grund bedeutsam ist. Seine Diagnosen benennen den Fehlerort nicht nur präzise, sondern tragen stabile Fehlercodes der Form `E0xxx` und ergänzen häufig konkrete, mit `help:` eingeleitete Korrekturvorschläge; ergänzt wird dies durch `cargo check` für eine schnelle Prüfung ohne vollständige Codegenerierung und durch den Linter `clippy` für eine weitergehende statische Analyse stilistischer und semantischer Schwachstellen [14].

Diese Eigenschaft gewinnt über den reinen Komfort beim Entwickeln hinaus an Bedeutung: Ein Compiler, der Fehler nicht nur meldet, sondern präzise lokalisiert und benennt, stellt ein hartes, automatisch und ohne menschliches Zutun prüfbares Ziel bereit, gegen das sich jede einzelne Codeänderung verifizieren lässt, sodass fehlerhafte Konstruktionen nicht erst zur Laufzeit, sondern bereits bei der Übersetzung

zurückgewiesen werden; genau das ist der Mechanismus, den maschinell erzeugter Quelltext benötigt. Wie sich diese Verbindung aus statischen Garantien und maschinenlesbarem Feedback methodisch nutzen lässt und in welchen Entwicklungsrahmen sie sich einfügt, behandelt der folgende Abschnitt (Abschnitt 3.4.2).

3.4.2 Agentengestützte Entwicklung (Agentic Coding)

4. Entwurf und Implementierung

4.1 Architektur

4.1.1 Architekturüberblick

4.1.2 Projektstruktur

4.1.3 Datenmodell der UDP Options

4.2 TLV-Parser und Serializer

4.2.1 Parsing-Algorithmus

4.2.2 Serialisierung

4.3 Option Checksum (OCS)

4.3.1 Berechnung

4.3.2 Validierung

4.4 Raw-Socket-Zugriff und Plattform-Spezifika

4.4.1 Socket-Erstellung und Paketempfang

4.4.2 Plattform-Spezifika (Linux und macOS)

4.4.3 Build-Konfiguration

5. Evaluation

5.1 Test- und Messkonzept

5.2 Beobachtbarkeit und Middlebox- Verträglichkeit auf realen Pfaden

5.3 Soll-Ist-Abgleich mit RFC 9868

6. Fazit

6.1 Beantwortung der Forschungsfragen

6.2 Ausblick

6.3 Schlusswort

Literaturverzeichnis

- [1] J. Postel. *User Datagram Protocol*. RFC 768. Internet Engineering Task Force, Aug. 1980. DOI: [10.17487/RFC0768](https://doi.org/10.17487/RFC0768). URL: <https://www.rfc-editor.org/info/rfc768>.
- [2] J. Touch. *Transport Options for UDP*. RFC 9868. Internet Engineering Task Force, Okt. 2025. DOI: [10.17487/RFC9868](https://doi.org/10.17487/RFC9868). URL: <https://www.rfc-editor.org/info/rfc9868>.
- [3] J. Touch. *Datagram Packetization Layer Path MTU Discovery for UDP Options*. RFC 9869. Internet Engineering Task Force, Okt. 2025. DOI: [10.17487/RFC9869](https://doi.org/10.17487/RFC9869). URL: <https://www.rfc-editor.org/info/rfc9869>.
- [4] V. Cerf, Y. Dalal und C. Sunshine. *Specification of Internet Transmission Control Program*. RFC 675. Internet Engineering Task Force, Dez. 1974. DOI: [10.17487/RFC0675](https://doi.org/10.17487/RFC0675). URL: <https://www.rfc-editor.org/info/rfc675>.
- [5] J. Iyengar und M. Thomson. *QUIC: A UDP-Based Multiplexed and Secure Transport*. RFC 9000. Internet Engineering Task Force, Mai 2021. DOI: [10.17487/RFC9000](https://doi.org/10.17487/RFC9000). URL: <https://www.rfc-editor.org/info/rfc9000>.
- [6] M. Bishop. *HTTP/3*. RFC 9114. Internet Engineering Task Force, Juni 2022. DOI: [10.17487/RFC9114](https://doi.org/10.17487/RFC9114). URL: <https://www.rfc-editor.org/info/rfc9114>.
- [7] R. Braden, D. Borman und C. Partridge. *Computing the Internet Checksum*. RFC 1071. Internet Engineering Task Force, Sep. 1988. DOI: [10.17487/RFC1071](https://doi.org/10.17487/RFC1071). URL: <https://www.rfc-editor.org/info/rfc1071>.
- [8] L. Eggert, G. Fairhurst und G. Shepherd. *UDP Usage Guidelines*. RFC 8085. Internet Engineering Task Force, März 2017. DOI: [10.17487/RFC8085](https://doi.org/10.17487/RFC8085). URL: <https://www.rfc-editor.org/info/rfc8085>.
- [9] M. Miller. *Trends, Challenges, and Strategic Shifts in the Software Vulnerability Mitigation Landscape*. Vortrag, BlueHat IL 2019, Tel Aviv. Microsoft Security Response Center, BlueHat IL 2019. 7. Feb. 2019. URL: <https://www.youtube.com/watch?v=PjbGojJnBZQ> (besucht am 31.05.2026).
- [10] The Chromium Project. *Memory safety*. The Chromium Project. URL: <https://www.chromium.org/Home/chromium-security/memory-safety/> (besucht am 31.05.2026).

- [11] National Security Agency. *Software Memory Safety. Cybersecurity Information Sheet*. National Security Agency (NSA). 10. Nov. 2022. URL: https://media.defense.gov/2022/Nov/10/2003112742/-1/-1/0/CSI_SOFTWARE_MEMORY_SAFETY.PDF (besucht am 31.05.2026).
- [12] The Rust Project. *Rust Programming Language*. 2025. URL: <https://www.rust-lang.org/> (besucht am 16.02.2026).
- [13] S. Klabnik und C. Nichols. *The Rust Programming Language*. 2025. URL: <https://doc.rust-lang.org/book/> (besucht am 16.02.2026).
- [14] The Rust Project. *The Rust Reference*. 2025. URL: <https://doc.rust-lang.org/reference/> (besucht am 16.02.2026).
- [15] ISO/IEC JTC 1/SC 22/WG14. *Programming languages: C*. N2310. Frei verfügbarer Arbeitsentwurf zu ISO/IEC 9899:2018 (C17). ISO/IEC JTC 1/SC 22/WG14. 2018. URL: <https://www.open-std.org/jtc1/sc22/wg14/www/docs/n2310.pdf> (besucht am 31.05.2026).
- [16] Zig Software Foundation. *Zig Language Reference*. Version 0.16.0. Zig Software Foundation. 2026. URL: <https://ziglang.org/documentation/master/> (besucht am 15.05.2026).
- [17] J. Sumner. *40 memory leaks and dozens of crashes fixed this way in bun v1.3.14*. X (vormals Twitter). 2026. URL: <https://x.com/jarredsumner/status/2055934488526619129> (besucht am 31.05.2026).

Eidesstattliche Erklärung

Ich versichere, dass ich die vorliegende Arbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe.

Alle Stellen der Arbeit, die wörtlich oder sinngemäß aus Veröffentlichungen oder aus anderweitigen fremden Quellen entnommen wurden, sind als solche kenntlich gemacht.

Die Arbeit wurde in gleicher oder ähnlicher Form noch keiner Prüfungsbehörde vorgelegt.

.....
Ort, Datum

.....
Unterschrift